# SLIDING WINDOW PROTOCOL

Exp.No: 07
Date : 24/08/26

**Aim:-**

Write a program to implement flow control for data link layer using SLIDING WINDOW PROTOCOL. Simulate the flow of frames from one node to another.

**Creating Sender program:-**

**Sender.py**

```
import time
import os
def write_file(filename, content):
    with open(filename, "w") as f:
        f.write(content)

def read_file(filename):
    if not os.path.exists(filename) or
os.path.getsize(filename) == 0:
        return None
    with open(filename, "r") as f:
        return f.read().strip()

def clear_file(filename):
    if os.path.exists(filename):
        open(filename, "w").close()
```

```
def main():
    print("=== SENDER PROGRAM ===")
    window_size = int(input("Enter Window
size (e.g., 3): "))
    message = input("Enter Text Message(
e.g., HELLO): ")

    total_frames = len(message)
    base = 0
    clear_file("Sender_Buffer.txt")
    clear_file("Receiver_Buffer.txt")

    while base < total_frames:
        buffer_content = ""
        current_window = []

        for i in range(base, min(base + window_
size, total_frames)):
            buffer_content += f"[{i}: {message[i]}]
            current_window.append(i)

        print(f"\n[Sender] Window frames: {current_
window}")
        write_file("Sender_Buffer.txt", buffer_content)

        print("[Sender] Waiting 10 seconds for Attacker
to modify 'Sender_Buffer.txt' now...")
        time.sleep(10)
```

```
rx_reply = None
for _ in range(5):
    rx_reply = read_file("Receiver_Buffer.txt")
    if rx_reply:
        break
    time.sleep(1)

if rx_reply:
    print(f"[Sender] Response received: {rx_reply}")
    try:
        status, ack_no = rx_reply.split(":")
        ackno = int(ack_no)
        if status == "ACK":
            print(f"[Sender] Success. Sliding window base frame {ack_no}.")
            base = ackno + 1
        elif status == "NACK":
            print(f"[Sender] Error noticed by Receiver! Retransmitting from frame {ack_no}...")
            base = ack_no
    except ValueError:
        print("[Sender] Bad response format. Retransmitting window...")
    clear_file("Receiver_Buffer.txt")
else:
    print("[Sender] Timeout! No Ack found
```

Print("\n[Sender] Completed! All
frames successfully transmitted and
acknowledged.")

write_file("Sender_Buffer.txt", D

if __name__ == "__main__":
    main()

Receiver.py

import time
import os

def write_file(filename, content):
    with open(filename, "w") as f:
        f.write(content)

def read_file(filname):
    if not os.path.exists(filename) or os.
path.getsize(filename) == 0:
        return None
    with open(filename, "r") as f:
        return f.read().strip()

def clear_file(filename):
    if os.path.exists(filename):
        open(filename, "w").close()

def main():
    print("=== RECEIVER PROGRAM ===")
    expected_frame = 0
    received_message = []

```
print("[Receiver] listening for incoming frames...")

while True:
    time.sleep(1)
    sender_data = read_file("sender_Buffer.txt")
    if not sender_data:
        continue
    if sender_data == "Done":
        print("\n[Receiver] sender completed transmission successfully!")
        break
    print(f"\n[Receiver] processing data from sender_Buffer.txt...")
    lines = sender_data.split("\n")
    error_found = False
    last_good_frame = expected_frame - 1

    for line in lines:
        if not line: continue
        try:
            frame_no, character = line.split(":")
            frame_no = int(frame_no)
            if frame_no == expected_frame and not error_found:
                print(f"[] Received frame{frame_no}: '{character}' -> [ERROR] Expected: {expected_frame}")
                error_found = True
        except ValueError:
```

```
        error_found = True

    if error_found:
        print(f"--> Replying with NACK: {expected_frame}")
        write_file("Receiver_Buffer.txt", f"NACK:{expected_frame}")
    else:
        if last_good_frame >= 0:
            print(f"--> Replying with ACK: {last_good_frame}")

    print(f"\nCurrent Received String: {''.join(received_message)}")
    clear_file("Sender_Buffer.txt")

if __name__ == "__main__":
    main()
```

Output:

```
== Reciver Program ==
Reciver waiting for incoming frames
== Sender program ==
Enter window size = 3
Enter text message = "HELLO"
Frames : [0, 1, 2]

Processing data from incoming frames
```

Received frame 0 : 'H' -> [OK]
Received frame 1 : 'e' -> [OK]
Received frame 2 : 'l' -> [OK]

----> Replying with ACK : 2
Current Received string : Hel

Result :

The program for Sliding window protocol has been executed successfully.